DevOps Technical Assessment - Completion Report (Revised)

Docker VMs, Ansible Automation and Hardened Nginx Reverse Proxy

Complete command reference: where to run, what it does, why it is used

How to Read This Report
1. Project Summary
2. Environment Setup
   2.1 Install WSL2
   2.2 Install Docker Desktop and connect it to Ubuntu
   2.3 Install the remaining tools
   2.4 Create the project folders
3. Section 4.1 - Docker Containers
   3.1 Generate the SSH key pair
   3.2 Write the Dockerfile
   3.3 Write the entrypoint script
   3.4 Create the custom network
   3.5 Build the VM image
   3.6 Run vm1, vm2 and vm3
   3.7 Verify containers, firewall and auto-restart
4. Section 4.2 - Hardened SSH
5. Section 4.3 - Local Domains and TLS
   5.1 Map the domain names (Windows hosts file)
   5.2 Generate the self-signed certificate
   5.3 First manual HTTPS test on vm1 (later replaced by Ansible)
6. Section 4.4 - Ansible Inventory and Configuration
7. Section 4.5 - Playbook, Roles and Vault
   7.1 Create the role folders
   7.2 Store the secret with ansible-vault
   7.3 Write the three roles
   7.4 Run the playbook twice (idempotency proof)
8. Section 4.6 - Dynamic Nginx Pages
9. Section 4.7 - Reverse Proxy, Load Balancing and Failover
   9.0 First manual test of the proxy (before it was templated)
   9.1 Test routing and load balancing
   9.2 Live failover test
10. Git and GitHub
   10.1 Create the repository and make the first push
   10.2 Project README
   10.3 Later updates, and adding this report
   10.4 Clean-up before submission
   10.5 Optional: package the project as a zip (without secrets)
11. Verification Summary
   11.1 Verified - output was observed during the build
   11.2 Still to run and capture (not yet recorded)
12. Errors Encountered and How They Were Resolved
   12.1 Docker build fails while downloading packages
   12.2 `docker kill` does not trigger the restart policy
   12.3 `no build stage in current context`
   12.4 Host cannot reach the container IPs after a restart
   12.5 `password_hash` fails on a vaulted value
   12.6 Reload handler fails: `Service is in unknown state`
   12.7 Idempotency broken: `changed=1` on every run
   12.8 Typing and pasting errors
   12.9 Playbook silently overwrote vm1's redirect and proxy config
   12.10 A private key was committed to the public repository
   12.11 One request took about 33 seconds during a failover test
13. Known Limitations and Honest Notes
   13.1 Passive health checks only
   13.2 Firewall enforcement is not yet verified
   13.3 Fixed password salt
   13.4 Host access through published ports
   13.5 Static IPs keep the upstream valid
   13.6 Items whose origin was not traced
14. Demonstration Video - Command Sequence
15. Repository and Submission

Appendix A - Command Glossary
Docker
SSH, network and web testing
Ansible
Git
Linux and Windows basics

How to Read This Report

Every command block follows the same pattern:

- **Run from** - the terminal and the folder (path) the command must be run in. Many commands only work from a specific folder, for example `ansible` commands must be run inside `/devops-assignment/ansible` because that is where `ansible.cfg` lives.
- The command block itself.
- **Why** - what the command does and why it was needed.

Terminals used

Name in this report	What it is
PowerShell (Admin)	Windows PowerShell opened with "Run as administrator". Used only to install or restart WSL2.
Ubuntu terminal	The Ubuntu tab in Windows Terminal (WSL2). **Every other command in this report is run here, as the root user**, so `sudo` is not needed.

Project root: `/devops-assignment` (the whole project and the GitHub repository live in this folder).

Placeholders: text in angle brackets such as `<your-password>` must be replaced with your own value. Real passwords are intentionally never written in this document.

Paste tip: in Windows Terminal paste with **Ctrl+Shift+V**. Copy commands from the document instead of retyping them; retyping caused several errors during the build (see section 12.8).

1. Project Summary

A three-container Docker "VM" infrastructure, configured end-to-end with Ansible, serving dynamic web pages behind a hardened Nginx reverse proxy with TLS, load balancing and automatic failover. Built at zero cost using only free and open-source tools.

Environment: Windows 11, WSL2 (Ubuntu 24.04), Docker Desktop (WSL2 backend), Windows Terminal.

Repository (public): `https://github.com/laxmikumb/devops-docker-ansible-assignment`

Architecture

```
Windows 11 host
 +-- WSL2 Ubuntu 24.04 : all commands run here, also the Ansible control node
 +-- Docker Desktop
      network "vmnet" 172.28.0.0/24
        vm1  172.28.0.11  proxy    host ports: 2201->22  8080->80  8443->443
        vm2  172.28.0.12  backend  host port : 2202->22
        vm3  172.28.0.13  backend  host port : 2203->22
      (all published ports are bound to 127.0.0.1 only)
```

Repository layout

```
/devops-assignment
  .gitignore
  gen_cert.sh                    broken leftover of a failed attempt - remove it (section 10.4)
  docker/   Dockerfile  entrypoint.sh  authorized_keys  vm_key.pub
  ansible/  ansible.cfg  inventory.ini  site.yml
            group_vars/all/vault.yml            (encrypted)
            roles/common/tasks/main.yml
            roles/ssh/tasks/main.yml            roles/ssh/handlers/main.yml
            roles/nginx/tasks/main.yml          roles/nginx/handlers/main.yml
            roles/nginx/templates/index.shtml.j2
            roles/nginx/templates/default.conf.j2
  nginx/certs/vm1.crt
  docs/     README.md  run1.log  run2.log  failover.log
            DevOps-Assignment-Completion-Report.pdf
```

Not committed on purpose: `docker/vm_key` (private SSH key), `ansible/.vault_pass.txt` (vault password), `nginx/certs/vm1.key` (TLS private key).

2. Environment Setup

2.1 Install WSL2

Run from: PowerShell (Admin), any folder.

```
wsl --install
```

Why: installs WSL2 and an Ubuntu distribution so Linux tools (Docker CLI, Ansible, OpenSSH) run on Windows. Restart Windows if asked. If Ubuntu is already installed the command simply opens it.

Related command used later when networking got stuck:

```
wsl --shutdown
```

Why: completely stops all WSL2 distributions so they start fresh next time. Note this is different from `wsl --install`; typing the wrong one is a common mix-up.

2.2 Install Docker Desktop and connect it to Ubuntu

Run from: Windows (graphical steps), then the Ubuntu terminal for the check.

1. Download and install Docker Desktop for Windows, keeping "Use WSL 2 instead of Hyper-V" ticked.
2. Open Docker Desktop, then **Settings > Resources > WSL integration**.
3. Switch **ON** the toggle next to **Ubuntu** (also keep "Enable integration with my default WSL distro" ticked), click **Apply**.

```
docker --version
```

Why: proves the Docker CLI inside Ubuntu can talk to Docker Desktop. If it says `command 'docker' could not be found in this WSL 2 distro`, the Ubuntu toggle in step 3 is still off.

2.3 Install the remaining tools

Run from: Ubuntu terminal, any folder.

```
apt update
apt install -y ansible openssl curl git nginx
ansible --version
git --version
```

Why:

- `apt update` refreshes the list of available packages.
- `apt install -y` installs: **ansible** (automation), **openssl** (certificates), **curl** (testing HTTP/HTTPS), **git** (version control), **nginx** (only needed inside the containers; the copy on the host is not used).
- `-y` answers "yes" automatically.
- The two `--version` commands confirm the tools are installed.

2.4 Create the project folders

Run from: Ubuntu terminal, any folder.

```
mkdir -p /devops-assignment/docker /devops-assignment/ansible /devops-assignment/nginx /devops-assignment/docs
cd /devops-assignment
git init
ls -la
```

Why:

- `mkdir -p` creates the folders (and does not complain if they exist).
- `cd /devops-assignment` moves into the project root; almost every later command is run from here.
- `git init` turns the folder into a Git repository (needed for GitHub in section 10).
- `ls -la` lists everything, including the hidden `.git` folder, to confirm it worked.

3. Section 4.1 - Docker Containers

3.1 Generate the SSH key pair

Run from: `/devops-assignment`

```
cd /devops-assignment
ssh-keygen -t ed25519 -f docker/vm_key -N ""
cp docker/vm_key.pub docker/authorized_keys
ls -la docker/
```

Why:

- `ssh-keygen` creates a key pair: `vm_key` (private, stays on the host) and `vm_key.pub` (public).
- `-t ed25519` selects a modern, strong key type. `-f docker/vm_key` sets the output path. `-N ""` means no passphrase (acceptable for a lab; use a passphrase in production).
- The public key is copied to `authorized_keys` because the Dockerfile copies that file into each VM so the key-only login works from the first boot.
- This must be done **before** building the image, otherwise the `COPY authorized_keys` step fails.

3.2 Write the Dockerfile

Run from: `/devops-assignment`. Create the file with `nano docker/Dockerfile`, paste the content below, save with Ctrl+O, Enter, then exit with Ctrl+X.

```
FROM ubuntu:22.04

RUN apt-get update \
    -o Acquire::Retries=5 \
    -o Acquire::http::No-Cache=true \
```

```
    -o Acquire::BrokenProxy=true \
    && apt-get install -y --fix-missing \
    -o Acquire::http::No-Cache=true \
    openssh-server sudo ufw iproute2 iputils-ping \
    python3 curl git nginx openssl vim net-tools \
    && mkdir -p /var/run/sshd

RUN useradd -m -s /bin/bash deployer \
    && mkdir -p /home/deployer/.ssh \
    && chmod 700 /home/deployer/.ssh \
    && echo "deployer ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deployer \
    && chmod 440 /etc/sudoers.d/deployer

COPY authorized_keys /home/deployer/.ssh/authorized_keys
RUN chmod 600 /home/deployer/.ssh/authorized_keys \
    && chown -R deployer:deployer /home/deployer/.ssh

RUN sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config \
    && sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config \
    && sed -i 's/^#\?PubkeyAuthentication.*/PubkeyAuthentication yes/' /etc/ssh/sshd_config

COPY entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh

EXPOSE 22 80 443
ENTRYPOINT ["/entrypoint.sh"]
```

Why each block:

- `FROM ubuntu:22.04` - the required Ubuntu base image. It **must be the first line** (see error 12.3).
- First `RUN` - installs OpenSSH server (remote login), `ufw` (firewall), `python3` (Ansible needs Python on the target), `nginx` (web server), and handy tools. The `-o Acquire::...` options make apt retry and bypass proxy caches, added after the download errors in 12.1.
- `useradd deployer` and the `sudoers.d` file - a normal user that logs in by key and can use `sudo` without a password. It has no password at all, so password login is impossible.
- `COPY authorized_keys ...` plus `chmod 600` / `chown` - installs the public key; SSH refuses keys with loose permissions.
- The three `sed` lines - harden `sshd_config`: password login off, root login off, key login on.
- `ENTRYPOINT` - the script that starts the firewall and services every time the container starts.

3.3 Write the entrypoint script

Run from: `/devops-assignment`. Create with `nano docker/entrypoint.sh`.

```
#!/bin/bash
set -e

ufw --force reset
ufw default deny incoming
ufw default allow outgoing
ufw allow from 172.28.0.0/24 to any port 22 proto tcp

if [ "$ROLE" = "proxy" ]; then
  ufw allow 80/tcp
  ufw allow 443/tcp
fi

ufw --force enable || true

service ssh start
nginx -g "daemon off;" &
wait
```

```
chmod +x docker/entrypoint.sh
cat docker/entrypoint.sh
```

Why:

- Containers have no systemd, so this script is what "boots" the VM: firewall first, then SSH, then Nginx.
- `ufw default deny incoming` is the default-deny policy required by the brief; `allow outgoing` keeps outbound traffic working.
- `ufw allow from 172.28.0.0/24 to any port 22` - SSH only from the Docker network.
- The `if [ "$ROLE" = "proxy" ]` block opens ports 80 and 443 only on vm1 (the container started with `-e ROLE=proxy`).
- `service ssh start` starts sshd. `nginx -g "daemon off;" &` runs Nginx in the background and `wait` keeps the script (PID 1) alive. **If Nginx dies, `wait` returns, the container exits, and Docker's restart policy brings it back** - this is how the crash test in 3.7 works.
- `chmod +x` makes the script executable; `cat` lets you confirm it was saved correctly.

3.4 Create the custom network

Run from: any folder.

```
docker network create --subnet=172.28.0.0/24 vmnet
docker network ls
docker network inspect vmnet
```

Why: the brief requires a custom network with a defined subnet, not Docker's auto-assigned default. `--subnet` fixes the address range, so each VM can be given a fixed IP. `network ls` shows `vmnet` exists; `network inspect` lists the subnet and, once containers run, each container's IP.

3.5 Build the VM image

Run from: `/devops-assignment/docker` (the folder that contains the Dockerfile, `authorized_keys` and `entrypoint.sh`).

```
cd /devops-assignment/docker
docker build --no-cache -t assignment-vm .
```

Why:

- `docker build` reads the Dockerfile and creates an image.
- `-t assignment-vm` names (tags) the image.
- The final `.` means "use the current folder as the build context", which is why the command must be run from `docker/`: `COPY authorized_keys` can only see files inside that folder.
- `--no-cache` forces every step to run again so no broken cached layer is reused.
- If apt downloads fail, see section 12.1. A first step that helped was `docker pull ubuntu:22.04`, which downloads the base image separately.

3.6 Run vm1, vm2 and vm3

Run from: any folder. Each command is **one single line** (long multi-line commands with backslashes broke when pasted, see 12.8).

```
docker run -d --name vm1 --hostname vm1 --network vmnet --ip 172.28.0.11 -p 127.0.0.1:2201:22 -p 127.0.0.1:8080:80 -p 127.0.0.1:8443:
cap-add=NET_ADMIN --restart unless-stopped -e ROLE=proxy assignment-vm
```

```
docker run -d --name vm2 --hostname vm2 --network vmnet --ip 172.28.0.12 -p 127.0.0.1:2202:22 --cap-add=NET_ADMIN --restart unless-st
-e ROLE=backend assignment-vm
```

```
docker run -d --name vm3 --hostname vm3 --network vmnet --ip 172.28.0.13 -p 127.0.0.1:2203:22 --cap-add=NET_ADMIN --restart unless-st
-e ROLE=backend assignment-vm
```

Option	Why it is used
`-d`	run in the background (detached)
`--name vm1`	container name required by the brief
`--hostname vm1`	the hostname shown inside the VM, and later displayed on the web page
`--network vmnet`	attach to the custom network
`--ip 172.28.0.11`	fixed IP (also keeps Nginx's upstream addresses valid after a restart, see 13.5)
`-p 127.0.0.1:2201:22`	publish container port 22 on the host's port 2201, **bound to 127.0.0.1 only** so nothing outside the PC can reach it (see 12.4 for why published ports were needed)
`--cap-add=NET_ADMIN`	permission to change firewall rules; `ufw` needs it
`--restart unless-stopped`	Docker restarts the container if it crashes or Windows reboots
`-e ROLE=proxy` / `backend`	tells `entrypoint.sh` whether to open ports 80/443

3.7 Verify containers, firewall and auto-restart

Run from: `/devops-assignment`

```
docker ps
docker logs vm2
docker exec vm1 ufw status verbose
docker exec vm2 ufw status verbose
```

Why:

- `docker ps` lists running containers (all three should be "Up") and the published ports.
- `docker logs vm2` shows what `entrypoint.sh` printed at start-up, including the UFW messages ("Default incoming policy changed to 'deny'", "Firewall is active and enabled on system startup").
- `docker exec <name> <command>` runs a command inside a running container. `ufw status verbose` prints the active firewall rules per VM, which the documentation section of the brief asks for. **Capture this output**; see limitation 13.2.

Crash test (auto-restart):

```
docker exec vm2 pkill -9 nginx
sleep 3
docker ps
docker inspect --format='{{.State.StartedAt}}' vm2
```

Why:

- `pkill -9 nginx` kills Nginx inside vm2. That ends the script's `wait`, so the container exits unexpectedly, which Docker treats as a **crash**.
- `sleep 3` gives Docker a moment to restart it.
- `docker ps` should show vm2 "Up" for only a few seconds; `StartedAt` shows a new timestamp, which is proof of the restart.
- `docker kill vm2` was deliberately **not** used: Docker treats it as the user stopping the container, so `unless-stopped` does not restart it (see 12.2). `docker ps -a` shows stopped containers if you need to check.

4. Section 4.2 - Hardened SSH

Run from: `/devops-assignment` (the key path `docker/vm_key` is relative to this folder; running from `docker/` gives "Identity file not accessible").

```
cd /devops-assignment
ssh -i docker/vm_key -p 2201 -o StrictHostKeyChecking=no deployer@localhost "hostname"
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -p 2201 deployer@localhost
ssh -i docker/vm_key -p 2201 root@localhost
```

Why each command and option:

- Command 1 - key login. `-i` chooses the private key, `-p 2201` the published port for vm1, `-o StrictHostKeyChecking=no` skips the "unknown host" question, and `"hostname"` runs one command and exits. **Expected output:** `vm1`.
- Command 2 - proves password login is refused. `PreferredAuthentications=password` and `PubkeyAuthentication=no` force a password attempt. **Expected:** `Permission denied (publickey)` with no password prompt, because the server only accepts keys.
- Command 3 - proves root login is refused even with a valid key. **Expected:** `Permission denied (publickey)`.

Key login to the other two VMs (needed for "SSH access via key to all VMs"):

```
ssh -i docker/vm_key -p 2202 -o StrictHostKeyChecking=no deployer@localhost "hostname"
ssh -i docker/vm_key -p 2203 -o StrictHostKeyChecking=no deployer@localhost "hostname"
```

Expected output: `vm2` **and** `vm3`**.** (vm1 results above were observed during the build; capture these two on video.)

5. Section 4.3 - Local Domains and TLS

5.1 Map the domain names (Windows hosts file)

Run from: Windows (graphical), then the Ubuntu terminal for the checks.

1. Press the Windows key, type **Notepad**, right-click it and choose **Run as administrator** (without admin rights Windows refuses to save the file).
2. In Notepad choose **File > Open**, paste this path, set the file-type drop-down to **All Files**, and open the file:

```
C:\Windows\System32\drivers\etc\hosts
```

3. Add these three lines at the bottom and save with Ctrl+S:

```
127.0.0.1  public.vm1.local
127.0.0.1  public.vm2.local
127.0.0.1  public.vm3.local
```

Why: the hosts file is the local "phone book" that maps names to addresses, so no paid domain is needed. The names point to `127.0.0.1` (not to the container IPs) because the host could not reach the container IPs directly on Docker Desktop (section 12.4); the published ports on `127.0.0.1` are used instead. This is documented as an assumption.

Verify (Ubuntu terminal, from `/devops-assignment`):

```
ping -c 2 public.vm1.local
ping -c 2 public.vm2.local
ping -c 2 public.vm3.local
ssh -i docker/vm_key -p 2202 -o StrictHostKeyChecking=no deployer@public.vm2.local "hostname"
ssh -i docker/vm_key -p 2203 -o StrictHostKeyChecking=no deployer@public.vm3.local "hostname"
```

Why: `ping -c 2` sends two packets and stops; replies from `127.0.0.1` prove the name resolves. The brief also asks to "SSH using these domain names", so the last two commands log in through the domain names (vm2 is on port 2202, vm3 on 2203). Expected output: `vm2` and `vm3`. The vm1 ping was observed during the build; capture the other four on video.

5.2 Generate the self-signed certificate

Run from: `/devops-assignment`

```
cd /devops-assignment
mkdir -p nginx/certs
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout nginx/certs/vm1.key -out nginx/certs/vm1.crt -subj /CN=public.vm1.local -
subjectAltName=DNS:public.vm1.local
ls -la nginx/certs/
```

Why each option:

- `req -x509` creates a self-signed certificate directly (no certificate authority needed, so no cost).
- `-nodes` leaves the private key unencrypted so Nginx can start without asking for a passphrase.
- `-days 365` validity of one year. `-newkey rsa:2048` creates a new 2048-bit RSA key.
- `-keyout` / `-out` set the key and certificate file paths.
- `-subj /CN=public.vm1.local` sets the certificate's name without interactive questions.
- `-addext subjectAltName=DNS:public.vm1.local` adds the Subject Alternative Name; modern clients ignore the CN alone, so the SAN is required for a valid certificate.
- `ls -la` should show `vm1.crt` (public) and `vm1.key` (private, mode 600).

Typing this long command by hand produced errors several times (a missing space, `subjectA1Name` instead of `subjectAltName`); always copy and paste it (12.8).

5.3 First manual HTTPS test on vm1 (later replaced by Ansible)

Run from: `/devops-assignment`

```
docker cp nginx/certs/vm1.crt vm1:/etc/ssl/certs/vm1.crt
```

```
docker cp nginx/certs/vm1.key vm1:/etc/ssl/private/vm1.key
```

```
cat > /tmp/vm1_nginx.conf << 'EOF'
server {
    listen 80;
    server_name public.vm1.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name public.vm1.local;

    ssl_certificate     /etc/ssl/certs/vm1.crt;
    ssl_certificate_key /etc/ssl/private/vm1.key;

    location / {
        ssi on;
        root /var/www/html;
        index index.html;
    }
}
EOF
docker cp /tmp/vm1_nginx.conf vm1:/etc/nginx/sites-available/default
docker exec vm1 nginx -t
docker exec vm1 nginx -s reload
```

```
echo "<h1>vm1 is working over HTTPS</h1>" > /tmp/test.html
docker cp /tmp/test.html vm1:/var/www/html/index.html
curl -Ik http://public.vm1.local:8080
curl -Ik https://public.vm1.local:8443
curl -k https://public.vm1.local:8443
```

Why:

- `docker cp <source> <container>:<path>` copies a file from the host into a container; here it places the certificate, key and config inside vm1.
- `cat > /tmp/vm1_nginx.conf << 'EOF' ... EOF` writes a file from the terminal; the quotes around `EOF` stop the shell from changing `$host`.
- `return 301 https://...` is the HTTP to HTTPS redirect. `nginx -t` checks the configuration for errors **before** applying it; `nginx -s reload` applies it without dropping connections.
- A first HTTPS request returned `403 Forbidden` because no web page existed yet; creating `index.html` fixed it.
- `curl -I` fetches only the headers, `-k` accepts the self-signed certificate (otherwise curl refuses it), and ports 8080 / 8443 are the host ports mapped to vm1's 80 / 443. Observed: `301 Moved Permanently` for HTTP, `200 OK` for HTTPS.
- This manual setup was a quick proof. In the final design Ansible deploys the Nginx configuration and generates its own certificate inside each VM at `/etc/nginx/ssl/` (section 7); the certificate created in 5.2 stays in the repository as the evidence of the OpenSSL step.

6. Section 4.4 - Ansible Inventory and Configuration

Run from: `/devops-assignment/ansible`

```
cd /devops-assignment/ansible
cat > inventory.ini << 'EOF'
[webservers]
vm1 ansible_host=127.0.0.1 ansible_port=2201
vm2 ansible_host=127.0.0.1 ansible_port=2202
vm3 ansible_host=127.0.0.1 ansible_port=2203

[webservers:vars]
ansible_user=deployer
ansible_python_interpreter=/usr/bin/python3
EOF

cat > ansible.cfg << 'EOF'
[defaults]
inventory = inventory.ini
private_key_file = ../docker/vm_key
host_key_checking = False
retry_files_enabled = False
EOF

cat inventory.ini
cat ansible.cfg
ansible all -m ping
```

Why:

- The inventory lists the three VMs. `ansible_host=127.0.0.1` and `ansible_port` point at the published SSH ports; `ansible_user=deployer` is the key-only user; `ansible_python_interpreter` tells Ansible where Python is inside the VMs.
- `ansible.cfg` removes the need to pass options on every run: `inventory` sets the inventory file, `private_key_file` sets the SSH key (relative to the `ansible` folder, hence `../docker/vm_key`), `host_key_checking = False` avoids host-key prompts in a lab.
- **Ansible reads `ansible.cfg` from the current folder**, so every `ansible` command in this report is run from `/devops-assignment/ansible`.
- `ansible all -m ping` runs Ansible's `ping` module on every host (an SSH + Python check, not an ICMP ping). **Expected: three `SUCCESS ... "ping": "pong"` results.**

7. Section 4.5 - Playbook, Roles and Vault

7.1 Create the role folders

Run from: `/devops-assignment/ansible`

```
cd /devops-assignment/ansible
mkdir -p roles/common/tasks roles/ssh/tasks roles/ssh/handlers roles/nginx/tasks roles/nginx/templates roles/nginx/handlers group_var
ls -la roles/
```

Why: roles split the work into reusable parts instead of one long file. `tasks/` holds the steps, `handlers/` holds actions triggered on change (reload, restart), `templates/` holds files with variables, and `group_vars/all` holds variables (including the vaulted secret) for every host.

7.2 Store the secret with ansible-vault

Run from: `/devops-assignment/ansible`

```
cd /devops-assignment/ansible
echo "<choose-a-vault-password>" > .vault_pass.txt
echo "vault_password_file = .vault_pass.txt" >> ansible.cfg
ansible-vault encrypt_string '<choose-a-strong-password>' --name 'vault_ansible_user_password' > group_vars/all/vault.yml
cat group_vars/all/vault.yml

cd /devops-assignment
echo ".vault_pass.txt" >> .gitignore
echo "docker/vm_key" >> .gitignore
echo "nginx/certs/vm1.key" >> .gitignore
echo "*.pem" >> .gitignore
cat .gitignore
```

Why:

- `.vault_pass.txt` holds the password used to encrypt and decrypt, so Ansible never has to ask for it. It is listed in `.gitignore` and must never be committed.
- `echo "..." >> ansible.cfg` **appends** a line (a single `>` would overwrite the whole file) so Ansible finds the vault password automatically.
- `ansible-vault encrypt_string` encrypts one value and prints a block that starts with `$ANSIBLE_VAULT;1.1;AES256`. That encrypted block is safe to commit. `--name` is the variable name used in the playbook.
- `.gitignore` tells Git to skip the private SSH key, the vault password, the TLS private key and any `.pem` file.
- **Do not write the real vault password or the secret value in documentation or chat**; anyone holding both the encrypted file and its password can decrypt it.

7.3 Write the three roles

Run from: `/devops-assignment/ansible`. Each `cat > file << 'EOF' ... EOF` creates one file. `cat file` afterwards confirms it.

Role `common` - user, sudo, packages

```
cat > roles/common/tasks/main.yml << 'EOF'
---
- name: Create ansible_user
  ansible.builtin.user:
    name: ansible_user
    password: "{{ vault_ansible_user_password | string | password_hash('sha512', 'fixedsalt123') }}"
    shell: /bin/bash
    state: present

- name: Grant ansible_user passwordless sudo via dedicated sudoers file
  ansible.builtin.copy:
    dest: /etc/sudoers.d/ansible_user
    content: "ansible_user ALL=(ALL) NOPASSWD:ALL\n"
    mode: "0440"
    validate: "visudo -cf %s"

- name: Install base packages
  ansible.builtin.apt:
    name: [curl, git, docker.io]
    state: present
    update_cache: true
EOF
```

Why: `user` creates `ansible_user` with a hashed password taken from the vault. `| string` and the fixed salt fix two errors (12.5 and 12.7). `copy` writes a **dedicated file in `/etc/sudoers.d/`** (required by the brief) with mode 0440, and `validate: visudo -cf %s` checks the syntax before saving so a typo cannot lock out sudo. `apt` installs curl, git and docker.io as the brief requires (the Docker engine is not used inside the containers; it is installed to satisfy the requirement).

Role `ssh` - keep SSH hardened

```
cat > roles/ssh/tasks/main.yml << 'EOF'
---
- name: Confirm password auth stays disabled
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: 'PasswordAuthentication no'
  notify: restart sshd

- name: Confirm root login stays disabled
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'
```

```
    notify: restart sshd
EOF

cat > roles/ssh/handlers/main.yml << 'EOF'
---
- name: restart sshd
  ansible.builtin.command: service ssh restart
EOF
```

Why: `lineinfile` makes sure exactly one correct line exists in `sshd_config`, so it only reports "changed" if the setting was wrong. `notify` triggers the handler only when a change happened. The handler runs `service ssh restart` directly because containers have no systemd (12.6).

Role `nginx` - install, certificate, page, site configuration

```
cat > roles/nginx/tasks/main.yml << 'EOF'
---
- name: Install nginx and openssl
  ansible.builtin.apt:
    name:
      - nginx
      - openssl
    state: present
    update_cache: true

- name: Create SSL directory
  ansible.builtin.file:
    path: /etc/nginx/ssl
    state: directory
    mode: "0755"

- name: Generate self-signed certificate
  ansible.builtin.command: >
    openssl req -x509 -nodes -days 365 -newkey rsa:2048
    -keyout /etc/nginx/ssl/selfsigned.key
    -out /etc/nginx/ssl/selfsigned.crt
    -subj "/CN=public.vm1.local"
  args:
    creates: /etc/nginx/ssl/selfsigned.crt

- name: Deploy dynamic hostname/time page (SSI)
  ansible.builtin.template:
    src: index.shtml.j2
    dest: /var/www/html/index.shtml
    mode: "0644"

- name: Deploy base site config
  ansible.builtin.template:
    src: default.conf.j2
    dest: /etc/nginx/sites-available/default
    mode: "0644"
  notify: reload nginx
EOF

cat > roles/nginx/handlers/main.yml << 'EOF'
---
- name: reload nginx
  ansible.builtin.command: nginx -s reload
EOF
```

Why: `apt` and `file` prepare Nginx and the certificate folder. The certificate task uses `args: creates:` - Ansible skips the command if that file already exists, which keeps the playbook **idempotent**. `template` fills a Jinja2 file with variables (such as `inventory_hostname`) and copies it to the VM. The handler reloads Nginx only when the site configuration changed, using `nginx -s reload` because the `service` module cannot query Nginx without systemd (12.6).

Template - dynamic page (`roles/nginx/templates/index.shtml.j2`)

```
cat > roles/nginx/templates/index.shtml.j2 << 'EOF'
<html>
<head><title>{{ inventory_hostname }}</title></head>
<body>
  <h1>Host: <!--#echo var="hostname" --></h1>
  <p>Server time: <!--#echo var="date_local" --></p>
</body>
</html>
EOF
```

Template - site configuration (`roles/nginx/templates/default.conf.j2`). The `{% if inventory_hostname == 'vm1' %}` condition gives vm1 the proxy configuration and gives vm2 and vm3 the plain page configuration (this fixed error 12.9).

```
cat > roles/nginx/templates/default.conf.j2 << 'EOF'
{% if inventory_hostname == 'vm1' %}
upstream app_backend {
    server vm2:80 max_fails=2 fail_timeout=5s;
    server vm3:80 max_fails=2 fail_timeout=5s;
}

server {
    listen 80;
    server_name public.vm1.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name public.vm1.local;
```

```
        ssl_certificate     /etc/nginx/ssl/selfsigned.crt;
        ssl_certificate_key /etc/nginx/ssl/selfsigned.key;

        location / {
            ssi on;
            root /var/www/html;
            index index.shtml;
        }

        location /vm2/ {
            proxy_pass http://vm2/;
            proxy_set_header Host $host;
        }

        location /vm3/ {
            proxy_pass http://vm3/;
            proxy_set_header Host $host;
        }

        location /app/ {
            proxy_pass http://app_backend/;
            proxy_next_upstream error timeout http_502 http_503 http_504;
            proxy_connect_timeout 2s;
        }
    }
    {% else %}
    server {
        listen 80 default_server;
        server_name _;
        ssi on;
        root /var/www/html;
        index index.shtml;
    }
    {% endif %}
    EOF
```

Playbook (`site.yml`)

```
    cat > site.yml << 'EOF'
    ---
    - hosts: webservers
      become: true
      roles:
        - common
        - ssh
        - nginx
    EOF
```

Why: `hosts: webservers` targets all three VMs, `become: true` runs the tasks with sudo, and `roles:` runs the three roles in order. This is the "roles, not one long flat file" structure the brief asks for.

7.4 Run the playbook twice (idempotency proof)

Run from: `/devops-assignment/ansible`

```
    cd /devops-assignment/ansible
    ansible-playbook site.yml | tee ../docs/run1.log
    ansible-playbook site.yml | tee ../docs/run2.log
```

Why:

- `ansible-playbook` runs the playbook. `| tee ../docs/run1.log` shows the output on screen **and** saves it to a file in the `docs` folder (`..` means "one folder up").
- Idempotent means running it again changes nothing. **The PLAY RECAP of the second run must show `changed=0` for vm1, vm2 and vm3.** Observed: `ok=11 changed=0 failed=0` on all three hosts.
- Note: the `run1.log` stored in the repository was captured after the VMs had already been configured, so it also shows `changed=0`. To capture a first run that shows real changes, recreate the containers (section 3.6) and run the playbook twice again.

8. Section 4.6 - Dynamic Nginx Pages

Run from: `/devops-assignment`

```
    docker exec vm2 curl -s http://localhost
    sleep 2
    docker exec vm2 curl -s http://localhost
    docker exec vm3 curl -s http://localhost
```

Why:

- The page is not a static file. Nginx's **SSI** (server-side includes, switched on by `ssi on;`) replaces `<!--#echo var="hostname" -->` and `<!--#echo var="date_local" -->` with the server's hostname and current time **every time the page is requested**.
- `docker exec vm2 curl -s http://localhost` runs `curl` inside vm2 (the backends have no published web port); `-s` hides the progress bar.
- Running it twice with `sleep 2` between shows a different time each run, which proves the content is generated per request. Observed: `Host: vm2` / `Host: vm3` with changing timestamps.

9. Section 4.7 - Reverse Proxy, Load Balancing and Failover

The proxy configuration is the vm1 branch of `default.conf.j2` (section 7.3), deployed by the playbook. How it works:

- `location /vm2/` and `/vm3/` forward the request to that VM (`proxy_pass http://vm2/;`). The names `vm2` and `vm3` resolve through Docker's built-in DNS on `vmnet`; traffic between containers stays inside Docker.
- `upstream app_backend` is the load-balanced pool; by default Nginx alternates between the servers (round-robin).
- `max_fails=2 fail_timeout=5s` is the **passive health check**: after 2 failed attempts a server is skipped for 5 seconds, then tried again automatically.
- `proxy_next_upstream error timeout http_502 http_503 http_504` retries the request on the other server instead of showing an error to the user. `proxy_connect_timeout 2s` limits how long Nginx waits to connect to a dead server.

9.0 First manual test of the proxy (before it was templated)

Run from: `/devops-assignment`. This first version was written by hand and copied into vm1 to prove the idea; it used the backend IP addresses and the certificate copied in 5.3. Section 7 later replaced it with the Ansible template (hostnames, certificate generated inside the VM).

```
        cat > /tmp/vm1_proxy.conf << 'EOF'
        upstream app_pool {
            server 172.28.0.12:80 max_fails=2 fail_timeout=5s;
            server 172.28.0.13:80 max_fails=2 fail_timeout=5s;
        }

        server {
            listen 80;
            server_name public.vm1.local;
            return 301 https://$host$request_uri;
        }

        server {
            listen 443 ssl;
            server_name public.vm1.local;

            ssl_certificate     /etc/ssl/certs/vm1.crt;
            ssl_certificate_key /etc/ssl/private/vm1.key;

            location / {
                ssi on;
                root /var/www/html;
                index index.shtml;
            }

            location /vm2/ {
                proxy_pass http://172.28.0.12:80/;
                proxy_set_header Host $host;
            }

            location /vm3/ {
                proxy_pass http://172.28.0.13:80/;
                proxy_set_header Host $host;
            }

            location /app/ {
                proxy_pass http://app_pool/;
                proxy_next_upstream error timeout http_502 http_503 http_504;
                proxy_connect_timeout 2s;
            }
        }
        EOF
        ls -la /tmp/vm1_proxy.conf
        docker cp /tmp/vm1_proxy.conf vm1:/etc/nginx/sites-available/default
        docker exec vm1 nginx -t
        docker exec vm1 nginx -s reload
```

Why: `ls -la /tmp/vm1_proxy.conf` confirms the file was really created (an earlier `docker cp` failed with "no such file" because the `cat > ... EOF` block had not been run first). `docker cp` puts the file where Nginx reads its site configuration, `nginx -t` must print "syntax is ok" and "test is successful", and `nginx -s reload` applies it without stopping Nginx.

9.1 Test routing and load balancing

Run from: `/devops-assignment`

```
        curl -k https://public.vm1.local:8443/vm2/
        curl -k https://public.vm1.local:8443/vm3/
        curl -k https://public.vm1.local:8443/app/
        curl -k https://public.vm1.local:8443/app/
```

Why: `-k` accepts the self-signed certificate. `/vm2/` must show vm2's page and `/vm3/` vm3's page. The two `/app/` requests must show different hosts (vm2 then vm3), proving load balancing. Observed: correct pages, and repeated `/app/` calls alternated vm2, vm3, vm2, vm3.

9.2 Live failover test

Run from: `/devops-assignment`. Use **one** terminal tab for all of the steps (`kill %1` only works in the same shell that started the background job).

```
        cd /devops-assignment
        ( while true; do curl -sk -o /dev/null -w "%{http_code} - %{time_total}s\n" https://public.vm1.local:8443/app/; sleep 1; done ) >
docs/failover.log 2>&1 &
        sleep 5
        tail -5 docs/failover.log
```

```
docker stop vm2
sleep 10
tail -10 docs/failover.log
```

```
docker start vm2
sleep 10
tail -15 docs/failover.log
```

```
kill %1
for i in 1 2 3 4 5 6; do curl -sk https://public.vm1.local:8443/app/ | grep Host; done
```

Why each step:

- The first block starts a loop in the background (`&`) that requests `/app/` once per second. `-s` is silent, `-k` ignores the certificate warning, `-o /dev/null` throws away the page, and `-w "%{http_code} - %{time_total}s\n"` prints only the HTTP status and the response time. `> docs/failover.log 2>&1` saves the output (and errors) to the evidence file; `tail` shows the latest lines.
- `docker stop vm2` takes one backend down while the loop keeps running. **Every line must still say `200`**, because Nginx moves the traffic to vm3. One request in this window takes about 2 seconds (observed 2.06 s to 2.20 s): that is the request that first tried the dead server and waited out `proxy_connect_timeout 2s` before being retried on vm3, so the user still gets a successful answer.
- `docker start vm2` brings it back. After its start-up finishes and `fail_timeout` expires, Nginx tries vm2 again by itself - **no `nginx -s reload` is needed**.
- `kill %1` stops the background loop. The final `for` loop sends six requests and prints only the `Host` lines: alternating `vm2` / `vm3` shows vm2 is back in rotation. Observed: `vm2, vm3, vm2, vm3, vm2, vm3`.

10. Git and GitHub

10.1 Create the repository and make the first push

Run from: the browser first, then the Ubuntu terminal in `/devops-assignment`.

1. In the browser open github.com, click **+ > New repository**, name it `devops-docker-ansible-assignment`, choose **Public**, leave "Add a README", ".gitignore" and "license" **unticked**, and click **Create repository**. Copy the repository URL.
2. Create a Personal Access Token: GitHub **Settings > Developer settings > Personal access tokens > Tokens (classic) > Generate new token (classic)**, set an expiry (7 days is enough), tick the **`repo`** scope, generate, and copy the `ghp_...` value at once (it is shown only once).

```
cd /devops-assignment
git config --global user.name "<Your Name>"
git config --global user.email "<your-email>"
git add .
git status
git commit -m "Initial commit: Docker VMs, Ansible automation, Nginx reverse proxy"
git remote add origin https://github.com/laxmikumb/devops-docker-ansible-assignment.git
git branch -M main
git push -u origin main
```

Why:

- The repository must be **public** (brief, section 7) and empty, otherwise the first push conflicts with GitHub's own starter files.
- GitHub no longer accepts the account password in Git. When `git push` asks for **Username**, type your GitHub username; when it asks for **Password**, paste the **token**. Never commit or share the token.
- `git config --global` sets the name and e-mail stored on commits (one-time setup). The "Committer: root ... configured automatically" message after a commit is only a warning.
- `git add .` stages every file that `.gitignore` does not exclude. **Always read `git status` before committing**: `docker/vm_key`, `.vault_pass.txt` and `nginx/certs/vm1.key` must not be listed. Reviewing it found two problems during the build: a typo file `docker/entryppoint.sh` (removed with `rm docker/entryppoint.sh`) and the TLS private key, which was removed from tracking with `git rm --cached nginx/certs/vm1.key` after adding it to `.gitignore`.
- `git commit -m` saves a snapshot with a message. `git remote add origin <url>` links the folder to the GitHub repository. `git branch -M main` renames the branch to `main`. `git push -u origin main` uploads it and remembers the link.

10.2 Project README

Run from: `/devops-assignment`

```
cd /devops-assignment
cat > docs/README.md << 'EOF'
# DevOps Technical Assessment - Implementation Documentation

## 1. Docker Setup
(text for this section)
EOF
ls -la docs/README.md

cat >> docs/README.md << 'EOF'

## 2. SSH Hardening
(text for this section)
EOF
ls -la docs/README.md
cat docs/README.md
```

Why: `docs/README.md` has eight sections: Docker setup, SSH hardening, domains and TLS, Ansible automation, dynamic pages, reverse proxy and load balancing, key assumptions, and a verification command list. It was built in chunks (`>` creates the file, `>>` appends) because one very long paste silently failed (12.8). `ls -la` after each chunk shows the file growing, and `cat` at the end lets you read the result. GitHub shows `docs/README.md` on the repository front page.

10.3 Later updates, and adding this report

Run from: `/devops-assignment`

```
        ls /mnt/c/Users/
        ls -la /mnt/c/Users/<WindowsUsername>/Downloads/ | grep -i devops
        cp "/mnt/c/Users/<WindowsUsername>/Downloads/<downloaded-file>.pdf" /devops-assignment/docs/DevOps-Assignment-Completion-Report.pdf
        ls -la docs/
        cd /devops-assignment
        git add .
        git status
        git commit -m "Update completion report"
        git push origin main
```

Why:

- `/mnt/c` is how WSL shows the Windows C: drive, so `ls /mnt/c/Users/` reveals your Windows user folder name.
- Browsers add suffixes such as `(1)` or `-final` when a file name already exists, so `grep -i devops` lists the matching downloads; pick the newest one by its date and size. An old copy was pushed by mistake once because the first match was an outdated download.
- `cp "<from>" <to>` copies the file into `docs/` (quotes are needed when the name contains spaces).
- The last four commands are the normal update cycle: stage, check, commit, push.

10.4 Clean-up before submission

Run from: `/devops-assignment`

```
        cd /devops-assignment
        git ls-files
        git rm gen_cert.sh
        git commit -m "Remove broken leftover script"
        git push origin main
```

Why:

- `git ls-files` lists every file Git tracks. Read it once: it must **not** contain `docker/vm_key`, `ansible/.vault_pass.txt`, `nginx/certs/vm1.key` or any `.pem` file.
- `gen_cert.sh` was written during a failed attempt to generate the certificate. The committed copy has a typo (`subjectAlName`) and broken line breaks, so it does not run. The certificate was created with the direct command in 5.2 and, inside the VMs, by Ansible. `git rm` deletes the file and stages the deletion in one step.

10.5 Optional: package the project as a zip (without secrets)

Run from: the filesystem root `/`.

```
        apt install -y zip
        cd /
        zip -r /mnt/c/Users/<WindowsUsername>/Downloads/devops-assignment.zip devops-assignment -x "devops-assignment/.git/*" "devops-
assignment/docker/vm_key" "devops-assignment/ansible/.vault_pass.txt" "devops-assignment/nginx/certs/vm1.key" "*.pem"
```

Why: `zip -r` bundles a folder recursively and `-x` lists patterns to leave out, here the Git history, the private SSH key, the vault password, the TLS key and any `.pem` file. The archive appears in your Windows Downloads folder.

11. Verification Summary

11.1 Verified - output was observed during the build

Requirement	Command	Result
Containers on custom subnet	`docker ps`, `docker network inspect vmnet`	vm1/vm2/vm3 up on 172.28.0.11 / .12 / .13
Firewall started	`docker logs vm2`	"Firewall is active and enabled on system startup"
Auto-restart on crash	`docker exec vm2 pkill -9 nginx`, `docker inspect ... StartedAt`	container back up, new StartedAt time
Key login (vm1)	`ssh -i docker/vm_key -p 2201 ... deployer@localhost "hostname"`	prints `vm1`
Password login rejected	`ssh -o PreferredAuthentications=password ...`	`Permission denied (publickey)`
Root login rejected	`ssh -i docker/vm_key -p 2201 root@localhost`	`Permission denied (publickey)`
Domain resolves	`ping public.vm1.local`	replies from 127.0.0.1
HTTP redirects to HTTPS	`curl -Ik http://public.vm1.local:8080`	`301 Moved Permanently`
HTTPS works	`curl -Ik https://public.vm1.local:8443`	`200 OK`
Ansible connectivity	`ansible all -m ping`	`pong` from vm1, vm2, vm3
Idempotency	`ansible-playbook site.yml` twice	second run `changed=0` on all hosts
Dynamic pages	repeated `curl` inside vm2 / vm3	hostname correct, time changes
Fixed-path routing	`curl -k .../vm2/`, `.../vm3/`	correct VM each time
Load balancing	repeated `curl -k .../app/`	alternates vm2 / vm3
Failover	background loop + `docker stop` / `start vm2`	all `200`, one ~2 s blip, vm2 rejoins without reload

11.2 Still to run and capture (not yet recorded)

Check	Command
Key login to vm2 and vm3	`ssh -i docker/vm_key -p 2202 ... deployer@localhost "hostname"` and port 2203
ping and SSH by domain name for vm2 and vm3	commands in section 5.1
Firewall rules per VM	`docker exec vm1 ufw status verbose`, `docker exec vm2 ufw status verbose`
Firewall enforcement test	commands in section 13.2

12. Errors Encountered and How They Were Resolved

12.1 Docker build fails while downloading packages

Symptom: `docker build` stopped during `apt-get install` with `E: Failed to fetch ... 400 Bad Request` on different packages (`libcbor0.8`, `iptables`). A separate attempt failed earlier with `server gave HTTP response to HTTPS client` while loading the `ubuntu:22.04` image metadata.

Likely cause: an unreliable network path on the Windows/WSL2 side that corrupted some larger downloads; the Quick Heal firewall on the PC may also have contributed. **The exact cause was not isolated**, because several changes were applied together.

What was done (Ubuntu terminal):

```
docker pull ubuntu:22.04
sudo tee /etc/resolv.conf <<'EOF'
nameserver 8.8.8.8
nameserver 1.1.1.1
EOF
cd /devops-assignment/docker
docker build --no-cache -t assignment-vm .
```

- `docker pull` downloads the base image on its own, which cleared the metadata error.
- `tee /etc/resolv.conf` switches WSL's DNS servers to Google and Cloudflare.
- The Dockerfile `apt-get` line was changed to use `-o Acquire::Retries=5`, `-o Acquire::http::No-Cache=true`, `-o Acquire::BrokenProxy=true` and `--fix-missing` (retry, and bypass proxy caches).
- The Quick Heal firewall (subscription expired) was switched off temporarily from its tray icon while building; re-enable it afterwards.
- Switching apt to the Azure Ubuntu mirror over HTTPS failed with `Connection refused` on port 443, so that change was reverted; the final Dockerfile uses the default mirror.

After these changes together the build succeeded. **Caution:** WSL regenerates `/etc/resolv.conf` when it restarts, so the DNS change is not permanent unless `generateResolvConf = false` is set under `[network]` in `/etc/wsl.conf`.

12.2 `docker kill` does not trigger the restart policy

Symptom: after `docker kill vm2` the container stayed in the `Exited (137)` state.

Cause: Docker treats `docker kill` and `docker stop` as a deliberate action by the user, and `--restart unless-stopped` means "restart unless the user stopped it". This is correct behaviour, not a bug. Killing PID 1 from inside the container with `kill -9 1` also does nothing, because Linux ignores signals sent from inside a PID namespace to its init process.

Fix: simulate a genuine crash by killing the service the container is waiting on:

```
docker exec vm2 pkill -9 nginx
sleep 3
docker inspect --format='{{.State.StartedAt}}' vm2
docker ps
```

The `wait` in `entrypoint.sh` ends, the container exits on its own, and Docker restarts it; the changed `StartedAt` time proves it. The containers also came back by themselves after a Windows restart.

12.3 `no build stage in current context`

Symptom: `docker build` failed immediately with `failed to solve: no build stage in current context`.

Cause: while editing in `nano`, a `RUN` line was pasted on the same line as `FROM ubuntu:22.04`, so `FROM` was no longer the first instruction.

Fix: recreate the file so `FROM` is line 1 and every instruction is on its own line, then check it before building:

```
cd /devops-assignment/docker
rm Dockerfile
nano Dockerfile
cat Dockerfile
```

12.4 Host cannot reach the container IPs after a restart

Symptom: `ssh ... deployer@172.28.0.11` and `ping 172.28.0.11` timed out, although `docker ps` showed the containers healthy. `ip route` on the Ubuntu side showed no route to `172.28.0.0/24`.

Likely cause: Docker Desktop on Windows runs Docker in its own hidden VM, and the routes of the custom bridge network are not always exposed to the WSL2 distribution. It worked right after the first run and failed after a restart; the exact trigger was not isolated. Restarting WSL (`wsl --shutdown`) and Docker Desktop did not bring the route back.

Diagnosis and fix:

```
ip route
ping -c 3 172.28.0.11
docker rm -f vm1 vm2 vm3
```

Then the three `docker run` commands from section 3.6 were used, now with `-p 127.0.0.1:<port>:<container port>` options. Access from the host goes through `localhost` ports, while container-to-container traffic (vm1 to vm2/vm3) keeps using the internal network and was never affected. Binding to `127.0.0.1` keeps the ports closed to the rest of the network. This is listed as an assumption in the README.

12.5 `password_hash` fails on a vaulted value

Symptom: `secret must be unicode or bytes, not AnsibleVaultEncryptedUnicode`.

Cause: the `password_hash` filter cannot take the special vault object directly.

Fix: convert it to a normal string first (the `| string` in the `common` role):

```
password: "{{ vault_ansible_user_password | string | password_hash('sha512') }}"
```

12.6 Reload handler fails: `Service is in unknown state`

Symptom: all tasks succeeded but the handler `reload nginx` failed on every VM.

Cause: the containers have no systemd, so Ansible's `service` module cannot read the service state.

Fix: run the commands directly in the handlers:

```
# roles/nginx/handlers/main.yml
- name: reload nginx
  ansible.builtin.command: nginx -s reload

# roles/ssh/handlers/main.yml
- name: restart sshd
  ansible.builtin.command: service ssh restart
```

12.7 Idempotency broken: `changed=1` on every run

Symptom: the task `Create ansible_user` reported `changed` on every run.

Cause: `password_hash` creates a new random salt each time, so the hash text is different on every run.

Fix: give the filter a fixed salt as the second argument, then run the playbook twice and check the recap:

```
password: "{{ vault_ansible_user_password | string | password_hash('sha512', 'fixedsalt123') }}"
```

```
cd /devops-assignment/ansible
ansible-playbook site.yml | tee ../docs/run1.log
ansible-playbook site.yml | tee ../docs/run2.log
```

The second run showed `changed=0`. Trade-off in 13.3.

12.8 Typing and pasting errors

Symptoms: `req: Use -help for summary` (a missing space, and `subjectA1Name` with the digit 1 instead of `subjectAltName`); `docker: invalid reference format` (a pasted command with `\` line breaks and `-restart` instead of `--restart`); big `cat > file << 'EOF'` blocks creating no file; `cd` typos such as `/devops-assignemnt`.

Cause: commands were typed by hand from the screen, and very long pastes in the old PowerShell window were incomplete.

Fix:

- Use the **Ubuntu tab in Windows Terminal** and paste with Ctrl+Shift+V.
- Write long `docker run` commands on one line.
- Build long files in pieces (`>` for the first, `>>` to append) and check the size after each piece:

```
cat > docs/README.md << 'EOF'
# Title and first sections
EOF
ls -la docs/README.md

cat >> docs/README.md << 'EOF'

## Next sections
EOF
ls -la docs/README.md
```

- Always confirm with `cat <file>` or `ls -la` before the next step, and use the Tab key to complete paths.

12.9 Playbook silently overwrote vm1's redirect and proxy config

Symptom: `curl -Ik http://public.vm1.local:8080` returned `200 OK` instead of `301`.

Cause: the Nginx site template was the same for all hosts. It served the page on port 80 with no redirect and had no `max_fails` / `fail_timeout` on the upstream, and it replaced the intended vm1 configuration. The `nginx` role file also contained extra tasks (SSL folder and certificate generation) whose origin was not traced; they were checked, found correct and kept.

Fix: rewrite `default.conf.j2` with the `{% if inventory_hostname == 'vm1' %}` condition (section 7.3), run the playbook, and re-test everything:

```
cat /devops-assignment/ansible/roles/nginx/templates/default.conf.j2
cd /devops-assignment/ansible
ansible-playbook site.yml
curl -Ik http://public.vm1.local:8080
curl -Ik https://public.vm1.local:8443
ansible-playbook site.yml | tee ../docs/run1.log
ansible-playbook site.yml | tee ../docs/run2.log
```

Result: `301` for HTTP, `200` for HTTPS, routing and load balancing correct, `changed=0` on the repeat runs. **Lesson:** after any playbook change, re-test every requirement, not only the PLAY RECAP.

12.10 A private key was committed to the public repository

Symptom: a file `privkey.pem` (not created by any step of this guide) was found in the project root.

Diagnosis:

```
cd /devops-assignment
ls -la privkey.pem
file privkey.pem
git log --all --oneline -- privkey.pem
```

`file` reported an OpenSSH private key and `git log` showed it was already in the first commit, so deleting it alone would not remove it from the public history.

Fix: delete it, ignore it, and replace the history with one clean commit:

```
rm privkey.pem
echo "privkey.pem" >> .gitignore
echo "*.pem" >> .gitignore
git add .
git checkout --orphan clean-main
git add -A
git commit -m "Clean history: remove exposed private key, fix vm1 Nginx redirect template"
git branch -D main
git branch -m main
git push origin main --force
```

Why: `--orphan` starts a branch with no history; deleting the old `main` and renaming this branch to `main` leaves only the clean commit; `--force` overwrites GitHub's copy. The key's origin was never found; any key that was ever public should be treated as compromised and not reused.

12.11 One request took about 33 seconds during a failover test

Symptom: in an early run one request in the curl loop took 33 s (still `200`). A clean re-run later showed only the expected ~2 s blips and every request `200`.

Cause: not diagnosed. It happened while the template problem in 12.9 was still present and vm2 was restarting. A possible explanation is that Nginx connected to vm2 while its start-up script was still resetting the firewall and waited for a reply; setting `proxy_read_timeout` and `proxy_send_timeout` lower on `/app/` would limit such a delay.

13. Known Limitations and Honest Notes

13.1 Passive health checks only

Active health checks (a prober that tests backends in the background) are an Nginx Plus feature. Open-source Nginx can only mark a server as failed when real requests to it fail (`max_fails`, `fail_timeout`). This satisfies "a passive or active health check" in the brief, but the first request after a failure can be slow (the ~2 s blip).

13.2 Firewall enforcement is not yet verified

The UFW rules are defined in `entrypoint.sh` and `docker logs` shows the firewall was activated. However, the backend rules only allow port 22 from `vmnet`; **port 80 is not explicitly allowed on vm2 and vm3, yet vm1 reaches them on port 80 through the proxy.** If UFW were enforcing the rules, that traffic would be blocked. So the rules are either not enforced inside Docker Desktop's kernel, or something else is allowing the traffic. Until `ufw status verbose` has been captured and tested, the firewall should be described as "configured", not as "proven".

Recommended follow-up. In `docker/entrypoint.sh` replace the `if` block with:

```
if [ "$ROLE" = "proxy" ]; then
  ufw allow 80/tcp
  ufw allow 443/tcp
else
  ufw allow from 172.28.0.11 to any port 80 proto tcp
fi
```

Rebuild, recreate the containers (section 3.6), re-run the playbook, and test:

```
cd /devops-assignment/docker
docker build -t assignment-vm .
```

```
docker rm -f vm1 vm2 vm3
# now run the three docker run commands from section 3.6, then:
cd /devops-assignment/ansible
ansible-playbook site.yml
docker exec vm2 ufw status verbose
curl -k https://public.vm1.local:8443/vm2/
docker exec vm3 curl -s --max-time 3 -o /dev/null -w "%{http_code}\n" http://vm2
```

Why: the new rule lets only vm1 (172.28.0.11) reach port 80 on the backends. The last command is the enforcement test: vm3 is not allowed to reach vm2, so `000` (timed out) means the firewall is working and `200` means it is not enforced. Repeat the failover test afterwards.

13.3 Fixed password salt

The fixed salt makes the hash repeatable (idempotent), but a constant salt is weaker than a random one. A better design sets the password only when the user is created (`update_password: on_create` on the `user` module) and keeps the random salt.

13.4 Host access through published ports

Names map to `127.0.0.1` and each VM is reached through its own published port, because the host could not route to `172.28.0.0/24` (12.4). On a native Linux host the names would map to the container IPs directly, as the brief describes.

13.5 Static IPs keep the upstream valid

Open-source Nginx resolves the names `vm2` and `vm3` in the `upstream` block only when it starts or reloads. Because every container has a fixed IP (`--ip`), vm2 comes back at the same address after `docker start`, which is why it rejoins without a reload. If addresses could change, a reload or a `resolver` setup would be needed.

13.6 Items whose origin was not traced

The stray `privkey.pem` (12.10), the extra SSL tasks in the `nginx` role (12.9) and the cause of the 33 s request (12.11).

14. Demonstration Video - Command Sequence

Run everything in the Ubuntu tab of Windows Terminal, in this order. Say the short sentence under each step while the output is on screen.

1. Containers, firewall and auto-restart (from `/devops-assignment`)

```
cd /devops-assignment
docker ps
docker exec vm1 ufw status verbose
docker exec vm2 ufw status verbose
docker exec vm2 pkill -9 nginx
sleep 3
docker ps
```

Say: "Three containers on the custom network. Firewall is default-deny. I crash Nginx in vm2 and Docker restarts the container by itself."

2. SSH to all VMs, password and root rejected

```
ssh -i docker/vm_key -p 2201 -o StrictHostKeyChecking=no deployer@localhost "hostname"
ssh -i docker/vm_key -p 2202 -o StrictHostKeyChecking=no deployer@localhost "hostname"
ssh -i docker/vm_key -p 2203 -o StrictHostKeyChecking=no deployer@localhost "hostname"
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no -p 2201 deployer@localhost
ssh -i docker/vm_key -p 2201 root@localhost
```

Say: "Key login works on all three VMs. A password attempt and a root login are both refused."

3. Domain names, HTTPS and redirect

```
ping -c 2 public.vm1.local
ping -c 2 public.vm2.local
ssh -i docker/vm_key -p 2202 -o StrictHostKeyChecking=no deployer@public.vm2.local "hostname"
curl -Ik http://public.vm1.local:8080
curl -Ik https://public.vm1.local:8443
```

Say: "The local domains resolve. HTTP gets a 301 redirect to HTTPS, and HTTPS answers 200 with the self-signed certificate."

4. Ansible connectivity

```
cd /devops-assignment/ansible
ansible all -m ping
```

Say: "Ansible reaches all three VMs using the key set in ansible.cfg."

5. Playbook and idempotency

```
ansible-playbook site.yml
ansible-playbook site.yml
```

Say: "The second run reports changed equals zero on every host, so the playbook is idempotent."

6. Nginx dynamic pages

```
docker exec vm2 curl -s http://localhost
```

```
sleep 2
docker exec vm2 curl -s http://localhost
docker exec vm3 curl -s http://localhost
```

Say: "Each page shows its own hostname and the time changes on every request, so it is generated dynamically."

7. Fixed-path routing and load balancing

```
cd /devops-assignment
curl -k https://public.vm1.local:8443/vm2/
curl -k https://public.vm1.local:8443/vm3/
curl -k https://public.vm1.local:8443/app/
curl -k https://public.vm1.local:8443/app/
```

Say: "vm1 proxies slash vm2 and slash vm3, and slash app alternates between vm2 and vm3."

8. Live failover (use one terminal tab for the whole step)

```
( while true; do curl -sk -o /dev/null -w "%{http_code} - %{time_total}s\n" https://public.vm1.local:8443/app/; sleep 1; done ) &
docker stop vm2
```

Wait about 8 seconds. Say: "vm2 is down and every request still returns 200, served by vm3."

```
docker start vm2
```

Wait about 8 seconds. Say: "vm2 is back and rejoins the rotation without a manual Nginx reload."

```
kill %1
for i in 1 2 3 4 5 6; do curl -sk https://public.vm1.local:8443/app/ | grep Host; done
```

Say: "The answers alternate between vm2 and vm3 again."

15. Repository and Submission

- **GitHub repository (public):** `https://github.com/laxmikumb/devops-docker-ansible-assignment`
- **Contents:** Docker files, Ansible inventory, roles and vaulted secret, Nginx configuration (as the Ansible template), documentation (`docs/README.md`), evidence logs (`run1.log` , `run2.log` , `failover.log`) and this report.
- **Not committed:** the private SSH key, the vault password file, the TLS private key and any `.pem` file (all in `.gitignore`).
- **Assessment form:** `https://forms.gle/SfX9aA8qUDXZgHs6A`

Appendix A - Command Glossary

A one-line meaning for every command used in this report. "Where" says the folder it normally runs from.

Docker

Command	What it does	Where
`docker build --no-cache -t NAME .`	builds an image from the Dockerfile in the current folder	`/devops-assignment/docker`
`docker pull IMAGE`	downloads an image from Docker Hub	any
`docker network create --subnet=... NAME`	creates a custom network with a fixed address range	any
`docker network ls` / `inspect NAME`	lists networks / shows one network's details and container IPs	any
`docker run -d --name ... IMAGE`	creates and starts a container in the background	any
`docker ps` / `docker ps -a`	lists running containers / all containers including stopped	any
`docker logs NAME`	prints what a container wrote at start-up	any
`docker exec NAME COMMAND`	runs a command inside a running container	any
`docker cp SOURCE NAME:PATH`	copies a file from the host into a container	folder where SOURCE exists
`docker inspect --format='{{.State.StartedAt}}' NAME`	prints one field of a container's details	any
`docker stop NAME` / `docker start NAME`	stops / starts a container (a deliberate stop, so no auto-restart)	any
`docker rm -f NAME`	force-removes a container	any

SSH, network and web testing

Command	What it does	Where
`ssh-keygen -t ed25519 -f PATH -N ""`	creates a key pair without a passphrase	`/devops-assignment`
`ssh -i KEY -p PORT USER@HOST "cmd"`	logs in with a key on a given port and runs one command	`/devops-assignment`
`ping -c 2 NAME`	sends 2 packets to test that a name resolves and answers	any

`ip route`	shows the routes the Ubuntu side knows	any
`curl -k URL`	requests a page and accepts a self-signed certificate	any
`curl -I` / `-s` / `-o /dev/null` / `-w "FORMAT"`	headers only / silent / discard the body / print status and timing	any
`openssl req -x509 -nodes ...`	creates a self-signed certificate and key	`/devops-assignment`
`ufw status verbose`	shows the firewall rules (run with `docker exec`)	any
`nginx -t` / `nginx -s reload`	checks the configuration / applies it without a restart (run with `docker exec`)	any

Ansible

Command	What it does	Where
`ansible --version`	shows the installed version	any
`ansible all -m ping`	tests SSH and Python on every host in the inventory	`/devops-assignment/ansible`
`ansible-playbook site.yml`	runs the playbook	`/devops-assignment/ansible`
`ansible-vault encrypt_string 'VALUE' --name 'VAR'`	encrypts one value into a vault block	`/devops-assignment/ansible`

Git

Command	What it does	Where
`git init`	turns a folder into a repository	`/devops-assignment`
`git config --global user.name / user.email`	sets the identity stored on commits	any
`git add .` / `git status`	stages all changes / shows what is staged	`/devops-assignment`
`git commit -m "MESSAGE"`	saves a snapshot	`/devops-assignment`
`git remote add origin URL`	links the folder to the GitHub repository	`/devops-assignment`
`git branch -M main`	renames the current branch to `main`	`/devops-assignment`
`git push -u origin main` / `git push origin main`	uploads commits to GitHub	`/devops-assignment`
`git rm FILE` / `git rm --cached FILE`	deletes a file and stages it / stops tracking but keeps the file	`/devops-assignment`
`git ls-files`	lists every tracked file	`/devops-assignment`
`git log --all --oneline -- FILE`	shows which commits touched a file	`/devops-assignment`
`git checkout --orphan NAME`	starts a branch with no history	`/devops-assignment`
`git branch -D main` / `git branch -m main`	deletes a branch / renames the current one	`/devops-assignment`
`git push origin main --force`	overwrites GitHub's history with the local one	`/devops-assignment`

Linux and Windows basics

Command	What it does	Where
`wsl --install` / `wsl --shutdown`	installs WSL / stops all WSL distributions	PowerShell (Admin)
`apt update` / `apt install -y PACKAGE`	refreshes package lists / installs packages	Ubuntu, any
`mkdir -p PATH`	creates folders, no error if they exist	any
`cd PATH`	changes the current folder; `cd ..` goes one level up	-
`ls -la`	lists files with sizes, dates and hidden files	any
`cat FILE`	prints a file	any
`cat > FILE << 'EOF' ... EOF` / `cat >> FILE << 'EOF' ... EOF`	writes / appends the text between the markers to a file	any
`echo "TEXT" >> FILE`	appends one line to a file (`>` would overwrite)	any
`cp SOURCE DESTINATION` / `rm FILE`	copies / deletes a file	any
`chmod +x FILE`	makes a script executable	any
`nano FILE`	text editor: Ctrl+O then Enter saves, Ctrl+X exits	any
`sleep N`	waits N seconds	any
`tail -N FILE`	prints the last N lines of a file	any
`kill %1`	stops background job number 1 of the current shell	same terminal tab
`file FILE`	tells what kind of file it is	any
`pkill -9 nginx`	force-kills processes named nginx (run with `docker exec`)	any
`sed -i 's/OLD/NEW/' FILE`	edits a file in place (used inside the Dockerfile)	-
`tee FILE`	writes input to a file and also shows it on screen	any
`zip -r OUT.zip FOLDER -x PATTERN`	creates a zip archive, leaving out the patterns	`/`

End of report.